OFFICIAL DEVELOPER HANDOVER CONTRACT

Suraj ERP — Backend API & Database Specification

Complete Data Structures, RESTful API Endpoints, JSON Schemas, and Business Guardrails

SYSTEM STACK	API ARCHITECTURE	BASE URL	DOCUMENT DATE
Next.js 16 / React 19 / TanStack Query	RESTful JSON over HTTP/HTTPS	`/api/v1`	August 2026 (v1.0 Production)

1. Global API Standards & Response Envelope

All backend services must communicate via standard RESTful JSON interfaces. Requests and responses must adhere to uniform status codes and envelopes.

Standard Headers

- `Content-Type: application/json`
- `Authorization: Bearer <JWT_TOKEN>`
- `Accept: application/json`

Unified Success Response Envelope (HTTP 200 / 201)

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": { ... },
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "totalPages": 8
  }
}
```

Unified Error Response Envelope (HTTP 400 / 401 / 403 / 404 / 422 / 500)

```
{
  "success": false,
  "message": "Validation failed / Resource constraint error",
  "error": {
    "code": "INVALID_INPUT",
    "details": [
      { "field": "gst", "issue": "Invalid GSTIN format. Expected 15 chars (e.g. 07AAACA123411Z5)."
    ]
  }
}
```

2. Authentication & User Management API

Manages user credentials, role-based access control (RBAC), and session tokens.

Database Entity: User (`users`)

FIELD NAME	DATA TYPE	CONSTRAINTS / ENUM VALUES	DESCRIPTION
`id`	String / UUID	Primary Key	Unique user identifier
`name`	String	Required	User's full name
`email`	String	Unique, Required	Login email address
`password`	String	Hashed (Argon2 / Bcrypt)	Encrypted password hash
`role`	String Enum	`ADMIN` `SALES_REP` `PURCHASE_OFFICER` `WAREHOUSE_MANAGER` `ACCOUNTANT`	Access permission role
`phone`	String	Optional	Contact phone number
`status`	String Enum	`ACTIVE` `INACTIVE`	Account status

API Endpoints

POST	**/api/v1/auth/login**	Authenticate & return JWT token
POST	**/api/v1/auth/register**	Create new system user account
GET	**/api/v1/auth/me**	Fetch current authenticated user profile
GET	**/api/v1/users**	List system users (?role=SALES_REP)
PUT	**/api/v1/users/:id**	Update user details or role permissions

3. CRM Module API

Manages Customers, Vendors, Sales Leads, Contacts, Deals, and Timeline Activity Logs.

Database Entity: Customers & Vendors (`crm_customers`)

```
{
  "id": "cust-101",
  "type": "Customer", // Enum: "Customer", "Vendor"
  "name": "Amitabh Sharma",
  "company": "Sharma Logistics Ltd.",
  "email": "amitabh@sharma-logistics.com",
  "phone": "+91 98765 43210",
  "gst": "09AAACH7409R1ZZ",
  "category": "Logistics",
  "status": "Active", // Enum: "Active", "Inactive"
  "outstanding": "₹42,850.00",
  "numericOutstanding": 42850.00,
  "creditLimit": "₹5,00,000.00",
  "numericCreditLimit": 500000.00,
  "assignedRep": "Sarah Jenkins",
  "billingAddress": "Plot 42, Transport Nagar, Kanpur, UP 208023",
  "shippingAddress": "Plot 42, Transport Nagar, Kanpur, UP 208023",
  "notes": "Key enterprise account for North India freight logistics."
}
```

Database Entity: Leads (`crm_leads`)

```
{
  "id": "lead-201",
  "name": "Vikram Malhotra",
  "company": "Apex Precision Tools",
  "email": "vikram@apexprecision.com",
  "phone": "+91 98111 22334",
  "source": "Direct Outreach", // Enum: "Direct Outreach", "Inbound Web Inquiry", "Trade Show Expo"
  "estimatedValue": "₹3,50,000.00",
  "numericValue": 350000.00,
  "stage": "Qualified", // Enum: "New", "Contacted", "Qualified", "Proposal", "Won", "Lost"
  "assignedRep": "Sarah Jenkins",
  "score": 85,
  "notes": "Interested in automated CNC machine components."
}
```

CRM API Endpoints

GET	**/api/v1/crm/customers**	Fetch customers/vendors (?type=Customer|Vendor&status=Active)
POST	**/api/v1/crm/customers**	Create customer or vendor party record
PUT	**/api/v1/crm/customers/:id**	Update customer profile & financial terms
DELETE	**/api/v1/crm/customers/:id**	Soft-delete customer record

GET	`/api/v1/crm/leads`	List leads (?stage=Qualified&search=Apex)
POST	`/api/v1/crm/leads`	Create sales prospect lead
PUT	`/api/v1/crm/leads/:id`	Update lead pipeline stage / details
GET	`/api/v1/crm/deals`	Fetch deals pipeline (?stage=Proposal)
POST	`/api/v1/crm/deals`	Create sales opportunity deal
GET	`/api/v1/crm/activities?entityId=:id`	Fetch timeline activity notes
POST	`/api/v1/crm/activities`	Log a call, meeting, or email activity note

4. Sales Module API

Handles Quotations, Sales Orders, Tax Invoices, Delivery Challans, Payments, and Returns.

> ⚠️ **Mandatory Business Rule: Single-Invoice Enforcement per Sales Order**
>
> Before saving an invoice (`type: "invoice"`), the backend **MUST** check if an active invoice linked to `salesOrderNo` already exists. If an invoice already exists, the backend must reject the request with HTTP `422 Unprocessable Entity` and message: *"An invoice has already been generated for this Sales Order."*

Database Entity: Sales Document (`sales_documents`)

```
{
  "id": "INV-2026-1441",
  "refNo": "INV-2026-1441",
  "type": "invoice", // Enum: "quotation", "sales_order", "invoice", "delivery_challan", "payment",
  "salesOrderNo": "SO-2026-1441",
  "poNumber": "PO-99420",
  "date": "2026-05-07",
  "status": "UNPAID", // Enum: "DRAFT", "PENDING", "APPROVED", "PAID", "UNPAID", "DELIVERED", "CANC
  "customer": "Acme Corp Pvt Ltd",
  "customerId": "cust-101",
  "gstin": "07AAACA123411Z5",
  "placeOfSupply": "07 - Delhi",
  "items": [
    {
      "description": "40 H.P. High Pressure Blower 2880 RPM 3700 CFM",
      "hsnSac": "84145930",
      "qty": 1,
      "unit": "Nos",
      "listPrice": 260000.00,
      "discRupees": 0.00,
      "taxPercent": 18.00
    }
  ],
  "subtotal": 260000.00,
  "taxTotal": 46800.00,
  "cgstAmount": 23400.00,
  "sgstAmount": 23400.00,
  "igstAmount": 0.00,
  "grandTotal": 306800.00
}
```

Sales API Endpoints

GET	**/api/v1/sales/documents?type=:type**	Fetch sales docs (quotation, sales_order, invoice, delivery_challan, payment)
POST	**/api/v1/sales/documents**	Save / create sales document
PUT	**/api/v1/sales/documents/:id**	Update sales document fields or line items
DELETE	**/api/v1/sales/documents/:id**	Delete sales document
GET	**/api/v1/sales/check-invoice-exists?salesOrderNo=:soNo**	Check if invoice exists for a Sales Order

5. Purchase Module API

Manages Requests For Order (RFOs), Purchase Bills, and Purchased Machinery Assets.

Database Entities: Purchase Domain (`purchase_records`)

```
// 1. Request For Order (RFO)
{
  "id": "RFO-2024-901",
  "refNo": "RFO-2024-901",
  "type": "rfo",
  "vendor": "Apex Industrial Solutions",
  "requestDate": "2024-10-14",
  "numericAmount": 145000.00,
  "department": "Toolroom & Precision Machining",
  "priority": "HIGH", // Enum: "NORMAL", "HIGH", "URGENT"
  "status": "PENDING APPROVAL"
}

// 2. Purchase Bill
{
  "id": "PB-2024-001",
  "type": "purchase_bill",
  "vendor": "Haas Automation India",
  "vendorInvoiceNo": "VINV-99120",
  "billDate": "2024-10-12",
  "dueDate": "2024-11-12",
  "numericAmount": 3850000.00,
  "status": "UNPAID"
}

// 3. Purchased Machinery Asset
{
  "id": "MAC-2024-881",
  "assetTag": "MAC-2024-881",
  "type": "purchased_machinery",
  "name": "CNC 5-Axis Milling Machine",
  "model": "Haas VF-4SS High Speed Vertical Center",
  "vendor": "Haas Automation India",
  "purchaseDate": "2024-01-15",
  "numericCost": 3850000.00,
  "location": "Bay A - Main Workshop",
  "status": "OPERATIONAL"
}
```

Purchase API Endpoints

GET	`/api/v1/purchase?type=rfo|purchase_bill|purchased_machinery`	List purchase records
POST	`/api/v1/purchase`	Create new purchase record (RFO, Bill, or Asset)
PUT	`/api/v1/purchase/:id`	Update purchase record
DELETE	`/api/v1/purchase/:id`	Delete purchase record

6. Inventory & Warehouse API

Manages Product Catalogs, Stock Movements, Low Stock Alerts, and Storage Bays in a single main factory hub.

> ⚙️ **Single Main Warehouse Setup**
>
> Suraj ERP operates on a single central factory warehouse setup: **Suraj Main Factory & Central Storage Hub** with 4 dedicated plant storage bays (Bay A - Raw Materials, Bay B - Machined Components, Bay C - Finished Goods, Bay D - Toolroom & Optics).

Database Entity: Product (`inventory_products`)

```
{
  "id": "PROD-001",
  "name": "Industrial Gear Set X12",
  "sku": "IG-1200-BL",
  "category": "Mechanical Parts",
  "warehouse": "Suraj Main Factory Warehouse (Bay A)",
  "stock": 850,
  "minReorder": 100,
  "unitPrice": 4250.00,
  "unit": "Units", // Enum: "Units", "Pcs", "Rolls", "Bags", "Boxes", "Kg"
  "status": "IN STOCK" // Enum: "IN STOCK", "LOW STOCK", "OUT OF STOCK"
}
```

Database Entity: Stock Movement Audit Log (`inventory_movements`)

```
{
  "id": "MOV-1001",
  "productId": "PROD-001",
  "productName": "Industrial Gear Set X12",
  "sku": "IG-1200-BL",
  "type": "STOCK IN", // Enum: "STOCK IN", "STOCK OUT"
  "quantity": "+250 Units",
  "numericQuantity": 250,
  "referenceNo": "PB-2024-001",
  "dateTime": "2026-05-24T11:32:00Z",
  "user": "R. Sharma"
}
```

Inventory API Endpoints

GET	**/api/v1/inventory/products**	List inventory items (?search=Gear&category=Mechanical)
POST	**/api/v1/inventory/products**	Add new inventory item / raw material
PUT	**/api/v1/inventory/products/:id**	Update item stock, price, or storage bay
GET	**/api/v1/inventory/movements**	Fetch stock movement audit history
POST	**/api/v1/inventory/receive-stock**	Post incoming stock (Increments item stock count)

POST	/api/v1/inventory/dispatch-stock	Post outgoing stock (Decrements item stock count)

7. Summary of Finance & Manufacturing Endpoints

Finance Endpoints

GET	/api/v1/finance/accounts	Fetch Chart of Accounts & General Ledger balances
GET	/api/v1/finance/vouchers	List journal / payment / receipt vouchers
POST	/api/v1/finance/vouchers	Post financial voucher entry
GET	/api/v1/finance/tax-summary	Fetch aggregated CGST, SGST, IGST liabilities & ITC

Manufacturing Endpoints

GET	/api/v1/manufacturing/bom	Fetch Bill of Materials (BOM) for finished products
GET	/api/v1/manufacturing/work-orders	Fetch factory production work orders
POST	/api/v1/manufacturing/work-orders	Issue new production work order

8. Key Architectural & Database Summary

1. **Database Recommendation**: PostgreSQL (or MySQL) with Prisma ORM / TypeORM or Drizzle ORM.
2. **Authentication Strategy**: JSON Web Tokens (JWT) stored in HTTP-only cookies or Bearer Authorization header.
3. **Auto Stock Movement Automation**: Fulfilling an Invoice or Delivery Challan triggers a background `STOCK OUT` stock movement; approving a Purchase Bill triggers a background `STOCK IN` stock movement.
4. **Soft Deletes & Audit Trails**: Never hard delete accounting or transaction records; set `isDeleted: true` and record `deletedAt` for full traceability.